AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Algorytm listy dwukierunkowej z zastosowaniem GitHub

Autor:
Hubert Muszyński

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Wstęp i cel projektu **4**

1.1. Wprowadzenie do problematyki 4

1.2. Cel i zakres projektu 4

1.3. Zastosowanie systemu kontroli wersji GitHub 5

1.4. Wymagania funkcjonalne i niefunkcjonalne 5

1.4.1. Wymagania funkcjonalne 5

1.4.2. Wymagania niefunkcjonalne 6

2. Ogólne założenia i architektura **7**

2.1. Założenia ogólne i struktura danych 7

2.2. Złożoność obliczeniowa operacji 7

2.3. Zastosowane wzorce projektowe 7

2.3.1. Wzorzec Fabryka (Factory Pattern) 8

2.3.2. Wzorzec Iterator (Iterator Pattern) 8

2.4. Scenariusze użycia i obieg danych 8

3. Projektowanie systemu **8**

3.1. Struktura plików projektu 8

3.2. Opis poszczególnych modułów i plików 9

3.2.1. Moduł Node 9

3.2.2. Moduł ListFactory 9

3.2.3. Moduł ListIterator 9

3.2.4. Moduł DoublyLinkedList 9

3.3. Specyfikacja interfejsów i funkcji 10

4. Implementacja programu **10**

4.1. Omówienie implementacji poszczególnych klas 10

4.2. Klasa Node 11

4.3. Klasa ListFactory 11

4.4. Klasa ListIterator 12

4.5. Szczegółowa analiza kluczowych fragmentów kodu 18

4.5.1. Wstawianie pod dowolny indeks (`insertAt`) 18
4.5.2. Bezpieczne zwalnianie pamięci w `popFront` i `clear` 18
4.6. Plik wykonawczy main.cpp 19

5. Wnioski i podsumowanie **20**

5.1. Podsumowanie wykonanych prac 20
5.2. Ocena realizacji celów i wymagań 20
5.3. Możliwości dalszego rozwoju projektu 21
5.4. Uwagi końcowe 21

Literatura **22**

Spis rysunków **22**

Spis tabel **23**

Spis listingów **24**

1. Wstęp i cel projektu

1.1. Wprowadzenie do problematyki

Dynamiczne struktury danych stanowią fundament współczesnej informatyki oraz inżynierii oprogramowania. W przeciwieństwie do statycznych tablic, których rozmiar musi być znany i zadeklarowany na etapie kompilacji lub alokacji pamięci, dynamiczne struktury danych pozwalają na elastyczne zarządzanie zasobami pamięci operacyjnej RAM w trakcie wykonywania programu (ang. *runtime*). Wśród dynamicznych struktur liniowych kluczową rolę odgrywają listy wiązane, a w szczególności **lista dwukierunkowa** (ang. *doubly linked list*).

Lista dwukierunkowa składa się z sekwencji węzłów (ang. *nodes*), z których każdy przechowuje wartość użytkową (właściwe dane) oraz dwa wskaźniki (adresy w pamięci): jeden wskazujący na następny element w sekwencji (`next`) oraz drugi wskazujący na element poprzedni (`prev`). Taka budowa umożliwia dwukierunkowe przechodzenie przez strukturę danych, co znacznie ułatwia realizację wielu operacji algorytmicznych, takich jak usuwanie elementów w czasie $O(1)$ przy posiadaniu wskaźnika na węzeł, czy też przeglądanie listy od końca do początku.

Celem niniejszego projektu jest zaprojektowanie, zaimplementowanie oraz przetestowanie pełnej, obiektowej biblioteki struktury listy dwukierunkowej w języku C++ z wykorzystaniem nowoczesnych standardów programowania obiektowego, wzorców projektowych oraz narzędzi kontroli wersji, takich jak system Git i platforma GitHub.

1.2. Cel i zakres projektu

Głównym celem realizowanego zadania jest stworzenie bezpiecznej, wydajnej i modułowej struktury danych reprezentującej listę dwukierunkową. W ramach projektu postawiono następujące cele szczegółowe:

- Implementacja dynamicznej alokacji i zwalniania pamięci bez wycieków (ang. *memory leaks*).
- Zastosowanie wzorca projektowego **Fabryka** (ang. *Factory Pattern*) do tworzenia węzłów struktury.
- Zaimplementowanie zewnętrznego **Iteratora** (ang. *Iterator Pattern*) pozwalającego na sekwencyjny dostęp do elementów listy bez ujawniania jej wewnętrznej reprezentacji.

- Zapewnienie pełnej obsługi sytuacji awaryjnych poprzez mechanizm wyjątków języka C++ (`std::out_of_range`, `std::underflow_error`).
- Zaprezentowanie właściwej organizacji kodu z podziałem na pliki nagłówkowe (`.h`) oraz pliki źródłowe (`.cpp`).
- Zarządzanie kodem źródłowym przy użyciu systemu kontroli wersji Git oraz repozytorium GitHub.

1.3. Zastosowanie systemu kontroli wersji GitHub

Współczesny proces wytwarzania oprogramowania wymaga stosowania zaawansowanych narzędzi do zarządzania kodem źródłowym. System Git pozwala na śledzenie historii zmian, tworzenie gałęzi (ang. *branches*) dla nowych funkcjonalności oraz bezproblemowe scalanie kodu. Platforma GitHub została wykorzystana w projekcie jako zdalne repozytorium do przechowywania kodu, dokumentowania zmian za pomocą precyzyjnych komunikatów zatwierdzeń (ang. *commit messages*) oraz archiwizacji wydań projektu.

Praca z repozytorium przebiegała zgodnie z modelem powszechnie stosowanym w projektach programistycznych, obejmującym:

1. Tworzenie niezależnych commitów dla każdej logicznej części kodu (np. dodanie klasy `Node`, implementacja iteratora, obsługa wyjątków).
2. Weryfikację poprawności działania przed wysłaniem zmian na serwer zdalny (`git push`).
3. Prowadzenie przejrzystego rejestru zmian (ang. *commit log*) ułatwiającego ewentualne debugowanie i analizę regresji.

1.4. Wymagania funkcjonalne i niefunkcjonalne

Przed przystąpieniem do fazy projektowania i implementacji zdefiniowano kompletny zestaw wymagań dla tworzonego systemu.

1.4.1. Wymagania funkcjonalne

Wymagania funkcjonalne określają konkretne zachowania i operacje, które biblioteka listy dwukierunkowej musi wykonywać. Zostały one zestawione w Tabeli 1.1.

Tab. 1.1. Zestawienie wymagań funkcjonalnych systemu

ID	Nazwa wymagania	Opis szczegółowy
WF-01	Dodawanie na początek	Możliwość wstawienia nowego elementu na początek listy (`pushFront`) w czasie stałym $O(1)$.
WF-02	Dodawanie na koniec	Możliwość dodania elementu na koniec listy (`pushBack`) w czasie stałym $O(1)$.
WF-03	Wstawianie pod indeks	Wstawienie elementu pod dowolny prawidłowy indeks (`insertAt`).
WF-04	Usuwanie z początku	Usunięcie elementu z głowy listy (`popFront`) z prawidło wym zwolnieniem pamięci.
WF-05	Usuwanie z końca	Usunięcie elementu z ogona listy (`popBack`) ze zwolnieniem pamięci.
WF-06	Usuwanie spod indeksu	Usuwanie węzła ze wskazanej pozycji (`removeAt`).
WF-07	Wyswietlanie przód/tył	Drukowanie zawartości listy w konsoli w obu kierunkach (`displayForward`, `displayBackward`).
WF-08	Iteracja po strukturze	Dostarczenie obiektu iteratora pozwalającego na poruszanie się przód/tył.
WF-09	Czyszczenie listy	Bezpieczne usunięcie wszystkich węzłów i zwolnienie pamięci (`clear`).

1.4.2. Wymagania niefunkcjonalne

Wymagania niefunkcjonalne określają cechy jakościowe oprogramowania:

- **Wydajność:** Kluczowe operacje dodawania/usuwania na krańcach struktury muszą charakteryzować się złożonością czasową $O(1)$.
- **Bezpieczeństwo pamięci:** Brak wycieków pamięci. Każda alokacja operatorem `new` musi posiadać odpowiadające zwolnienie operatorem `delete`.
- **Niezawodność:** Odporność na błędne dane wejściowe poprzez zgłaszanie standardowych wyjątków języka C++.
- **Czytelność kodu:** Zastosowanie konwencji nazewnictwa, komentarzy oraz podziału na moduły.

2. Ogólne założenia i architektura

2.1. Założenia ogólne i struktura danych

Lista dwukierunkowa charakteryzuje się tym, że każdy jej element (węzeł) posiada powiązanie zarówno z elementem go poprzedzającym, jak i następującym po nim. Zewnętrzny interfejs listy przechowuje wskaźnik na początek listy (`head`), wskaźnik na koniec listy (`tail`) oraz aktualną liczbę elementów (`size`).

Dzięki przechowywaniu wskaźnika `tail`, operacje wstawiania oraz usuwania elementów na końcu listy mogą odbywać się bez konieczności przechodzenia przez całą strukturę od początku, co redukuje ich złożoność czasową ze złożoności liniowej $O(n)$ do złożoności stałej $O(1)$.

2.2. Złożoność obliczeniowa operacji

W Tabeli 2.1 przedstawiono teoretyczną złożoność czasową poszczególnych operacji realizowanych przez zaimplementowaną listę dwukierunkową.

Tab. 2.1. Porównanie złożoności obliczeniowej operacji na liście dwukierunkowej

Operacja	**Złożoność średnia**	**Złożoność najgorsza**
Dostęp po indeksie	$O(n)$	$O(n)$
Wyszukiwanie wartości	$O(n)$	$O(n)$
Wstawianie na początek (`pushFront`)	$O(1)$	$O(1)$
Wstawianie na koniec (`pushBack`)	$O(1)$	$O(1)$
Wstawianie wewnątrz (`insertAt`)	$O(n)$	$O(n)$
Usuwanie z początku (`popFront`)	$O(1)$	$O(1)$
Usuwanie z końca (`popBack`)	$O(1)$	$O(1)$
Usuwanie wewnątrz (`removeAt`)	$O(n)$	$O(n)$

2.3. Zastosowane wzorce projektowe

W projekcie zdecydowano się na świadome zastosowanie dwóch klasycznych wzorców projektowych ułatwiających utrzymanie kodu oraz rozdzielenie odpowiedzialności pomiędzy poszczególnymi klasami.

2.3.1. Wzorzec Fabryka (Factory Pattern)

Klasa `ListFactory` pełni rolę wytwórczą dla obiektów typu `Node`. Enkapsulacja procesu tworzenia obiektu pozwala na:

- Centralizację logiki przydzielania pamięci dla węzłów.
- Łatwe rozbudowanie procesu inicjalizacji węzłów w przyszłości (np. dodanie logowania, alokatorów pamięci, monitorowania wycieków).
- Odizolowanie klasy `DoublyLinkedList` od bezpośrednich wywołań operatora `new`.

2.3.2. Wzorzec Iterator (Iterator Pattern)

Wzorzec Iterator udostępniany przez klasę `ListIterator` umożliwia bezpieczne sekwencyjne przechodzenie po elementach struktury danych bez ujawniania jej wewnętrznej budowy wskaźnikowej. Zaletami takiego rozwiązania są:

- Ukrycie bezpośrednich wskaźników `Node*` przed użytkownikiem końcowym aplikacji.
- Ochrona spójności struktury (użytkownik nie może przypadkowo zerwać więzów wskaźnikowych między węzłami).
- Ujednolicony interfejs przeglądania elementów za pomocą metod `hasNext()`, `next()`, `hasPrev()` oraz `prev()`.

2.4. Scenariusze użycia i obieg danych

Przepływ danych w aplikacji opiera się na wykonywaniu operacji przez klasę główną `main`, która tworzy instancję `DoublyLinkedList`. Klasa listy w razie potrzeby wstawienia danych korzysta z `ListFactory`, otrzymując gotowe wskaźniki na `Node`. Odczyt i przeglądanie danych realizowane są poprzez przemieszczanie stanu wewnętrznego wskaźnika w klasie `ListIterator`.

3. Projektowanie systemu

3.1. Struktura plików projektu

Projekt został podzielony na niezależne moduły zgodnie z zasadą pojedynczej odpowiedzialności (ang. *Single Responsibility Principle*). Wykaz wszystkich plików wcho-

dzących w skład systemu przedstawiono poniżej:

- `Node.h` / `Node.cpp` – definicja i implementacja pojedynczego węzła listy.
- `ListFactor.h` / `ListFactor.cpp` – klasa fabryki odpowiedzialna za kreację węzłów.
- `ListIterator.h` / `ListIterator.cpp` – implementacja zewnętrznego iteratora do poruszania się po liście.
- `DoublyLinkedList.h` / `DoublyLinkedList.cpp` – główna klasa zarządzająca strukturą listy dwukierunkowej.
- `main.cpp` – plik zawierający funkcję główną, scenariusze testowe i weryfikację poprawności działania.

3.2. Opis poszczególnych modułów i plików

3.2.1. Moduł Node

Plik `Node.h` definiuje podstawową strukturę przechowywania danych. Składa się z publicznych pól `data` (wartość typu `int`), `next` (wskaźnik na kolejny węzeł) oraz `prev` (wskaźnik na poprzedni węzeł). Konstruktor klasy przyjmuje wartość całkowitą i inicjalizuje wskaźniki wartościami `nullptr`.

3.2.2. Moduł ListFactory

Klasa `ListFactory` udostępnia statyczną metodę `createNode(int value)`, która odpowiada za dynamiczną alokację pamięci dla nowego obiektu `Node` na stercie (ang. *heap*) i zwrócenie do niego wskaźnika.

3.2.3. Moduł ListIterator

Klasa `ListIterator` przechowuje wskaźnik na bieżący węzeł (`current`). Udostępnia metody pozwalające na sprawdzanie dostępności kolejnych/poprzednich elementów oraz na pobieranie i modyfikację wartości przechowywanej w bieżącym węźle.

3.2.4. Moduł DoublyLinkedList

Główna klasa systemu zarządzająca wskaźnikami `head` i `tail`. Odpowiada za zachowanie spójności struktury danych przy wszystkich operacjach dodawania, usuwania, wyszukiwania oraz czyszczenia danych.

3.3. Specyfikacja interfejsów i funkcji

W Tabeli 3.1 opisano zestaw metod udostępnianych przez klasę `DoublyLinkedList`.

Tab. 3.1. Specyfikacja interfejsu klasy `DoublyLinkedList`

Metoda	Typ zwracany	Opis działania
`pushFront(int value)`	`void`	Wstawia nową wartość na początek listy.
`pushBack(int value)`	`void`	Wstawia nową wartość na koniec listy.
`insertAt(size_t idx, int val)`	`void`	Wstawia wartość pod podany indeks.
`popFront()`	`void`	Usuwa pierwszy element listy.
`popBack()`	`void`	Usuwa ostatni element listy.
`removeAt(size_t index)`	`void`	Usuwa element znajdujący się pod podanym indeksem.
`displayForward()`	`void`	Drukuje elementy od głowy do ogona.
`displayBackward()`	`void`	Drukuje elementy od ogona do głowy.
`clear()`	`void`	Zwalnia pamięć wszystkich elementów.
`getSize()`	`size_t`	Zwraca aktualną liczbę elementów w liście.
`getIterator()`	`ListIterator`	Zwraca iterator ustawiony na głowę listy.

4. Implementacja programu

4.1. Omówienie implementacji poszczególnych klas

W niniejszym rozdziale zaprezentowano pełny kod źródłowy poszczególnych modułów wraz ze szczegółową analizą kluczowych algorytmów oraz wybranych fragmentów kodu.

4.2. Klasa Node

Listing 1 oraz Listing 2 przedstawiają definicję i implementację klasy `Node`.

```
#ifndef NODE_H
#define NODE_H

class Node {
public:
    int data;
    Node* next;
    Node* prev;

    Node(int val);
};

#endif
```

Listing 1. Plik nagłówkowy Node.h

```
#include "Node.h"

Node::Node(int val) : data(val), next(nullptr), prev(nullptr) {}
```

Listing 2. Plik źródłowy Node.cpp

Konstruktor klasy `Node` wykorzystuje listę inicjalizacyjną do przypisania wartości pola `data` oraz do ustawienia wskaźników `next` i `prev` na wartość `nullptr`, co zapobiega powstawaniu tzw. "dzikich wskaźników"(ang. *dangling pointers*).

4.3. Klasa ListFactory

Fabryka węzłów zaimplementowana w plikach `ListFactor.h` oraz `ListFactor.cpp` odpowiada za kreację nowych obiektów.

```
#ifndef LIST_FACTORY_H
#define LIST_FACTORY_H

#include "Node.h"

class ListFactory {
public:
    static Node* createNode(int value);
};

#endif
```

Listing 3. Plik nagłówkowy ListFactor.h

```cpp
#include "ListFactor.h"

Node* ListFactory::createNode(int value) {
    return new Node(value);
}
```

Listing 4. Plik źródłowy ListFactor.cpp

4.4. Klasa ListIterator

Klasa `ListIterator` umożliwia poruszanie się po liście w obu kierunkach.

```cpp
#ifndef LIST_ITERATOR_H
#define LIST_ITERATOR_H

#include "Node.h"

class ListIterator {
private:
    Node* current;
public:
    ListIterator(Node* start);
    bool hasNext() const;
    bool hasPrev() const;
    int next();
    int prev();
    int get() const;
    void set(int val);
    Node* getCurrent() const;
};

#endif
```

Listing 5. Plik nagłówkowy ListIterator.h

```cpp
#include "ListIterator.h"
#include <stdexcept>

ListIterator::ListIterator(Node* start) : current(start) {}

bool ListIterator::hasNext() const {
    return current != nullptr && current->next != nullptr;
}

bool ListIterator::hasPrev() const {
    return current != nullptr && current->prev != nullptr;
```

```
12 }
13
14 int ListIterator::next() {
15     if (!hasNext()) throw std::out_of_range("Brak nastepnego
       elementu.");
16     current = current->next;
17     return current->data;
18 }
19
20 int ListIterator::prev() {
21     if (!hasPrev()) throw std::out_of_range("Brak poprzedniego
       elementu.");
22     current = current->prev;
23     return current->data;
24 }
25
26 int ListIterator::get() const {
27     if (!current) throw std::runtime_error("Iterator wskazuje na
       nullptr.");
28     return current->data;
29 }
30
31 void ListIterator::set(int val) {
32     if (!current) throw std::runtime_error("Iterator wskazuje na
       nullptr.");
33     current->data = val;
34 }
35
36 Node* ListIterator::getCurrent() const {
37     return current;
38 }
39 \end{lstlisting">
40
41 \subsection{G Ć wna klasa DoublyLinkedList}
42 Poni ej przedstawiono kod pliku nag Ć wkowego oraz pliku
       wykonawczego g Ć wnej struktury danych.
43
44 \begin{lstlisting}[language=C++, caption={Plik nag Ć wkowy
       DoublyLinkedList.h}, label={lst:list_h}]
45 #ifndef DOUBLY_LINKED_LIST_H
46 #define DOUBLY_LINKED_LIST_H
47
48 #include <cstddef>
49 #include "Node.h"
50 #include "ListIterator.h"
```

```
51
52 class DoublyLinkedList {
53 private:
54     Node* head;
55     Node* tail;
56     size_t size;
57
58 public:
59     DoublyLinkedList();
60     ~DoublyLinkedList();
61
62     void pushFront(int value);
63     void pushBack(int value);
64     void insertAt(size_t index, int value);
65
66     void popFront();
67     void popBack();
68     void removeAt(size_t index);
69
70     void displayForward() const;
71     void displayBackward() const;
72
73     void clear();
74     size_t getSize() const;
75
76     ListIterator getIterator() const;
77     ListIterator getReverseIterator() const;
78 };
79
80 #endif
```

Listing 6. Plik źródłowy ListIterator.cpp

```
1 #include "DoublyLinkedList.h"
2 #include "ListFactor.h"
3 #include <iostream>
4 #include <stdexcept>
5
6 DoublyLinkedList::DoublyLinkedList() : head(nullptr), tail(nullptr)
      , size(0) {}
7
8 DoublyLinkedList::~DoublyLinkedList() {
9     clear();
10 }
11
12 void DoublyLinkedList::pushFront(int value) {
```

```
13      Node* newNode = ListFactory::createNode(value);
14      if (!head) {
15          head = tail = newNode;
16      } else {
17          newNode->next = head;
18          head->prev = newNode;
19          head = newNode;
20      }
21      size++;
22  }
23
24  void DoublyLinkedList::pushBack(int value) {
25      Node* newNode = ListFactory::createNode(value);
26      if (!tail) {
27          head = tail = newNode;
28      } else {
29          tail->next = newNode;
30          newNode->prev = tail;
31          tail = newNode;
32      }
33      size++;
34  }
35
36  void DoublyLinkedList::insertAt(size_t index, int value) {
37      if (index > size) throw std::out_of_range("Indeks poza zakresem
        .");
38      if (index == 0) {
39          pushFront(value);
40          return;
41      }
42      if (index == size) {
43          pushBack(value);
44          return;
45      }
46
47      Node* current = head;
48      for (size_t i = 0; i < index; i++) {
49          current = current->next;
50      }
51
52      Node* newNode = ListFactory::createNode(value);
53      Node* prevNode = current->prev;
54
55      prevNode->next = newNode;
56      newNode->prev = prevNode;
```

```
57
58      newNode->next = current;
59      current->prev = newNode;
60
61      size++;
62  }
63
64  void DoublyLinkedList::popFront() {
65      if (!head) throw std::underflow_error("Lista jest pusta.");
66      Node* temp = head;
67      head = head->next;
68      if (head) {
69          head->prev = nullptr;
70      } else {
71          tail = nullptr;
72      }
73      delete temp;
74      size--;
75  }
76
77  void DoublyLinkedList::popBack() {
78      if (!tail) throw std::underflow_error("Lista jest pusta.");
79      Node* temp = tail;
80      tail = tail->prev;
81      if (tail) {
82          tail->next = nullptr;
83      } else {
84          head = nullptr;
85      }
86      delete temp;
87      size--;
88  }
89
90  void DoublyLinkedList::removeAt(size_t index) {
91      if (index >= size) throw std::out_of_range("Indeks poza
        zakresem.");
92      if (index == 0) {
93          popFront();
94          return;
95      }
96      if (index == size - 1) {
97          popBack();
98          return;
99      }
100
```

```
101     Node* current = head;
102     for (size_t i = 0; i < index; i++) {
103         current = current->next;
104     }
105 
106     current->prev->next = current->next;
107     current->next->prev = current->prev;
108     delete current;
109     size--;
110 }
111 
112 void DoublyLinkedList::displayForward() const {
113     Node* current = head;
114     std::cout << "Lista (przod): ";
115     while (current) {
116         std::cout << current->data << " ";
117         current = current->next;
118     }
119     std::cout << std::endl;
120 }
121 
122 void DoublyLinkedList::displayBackward() const {
123     Node* current = tail;
124     std::cout << "Lista (tyl): ";
125     while (current) {
126         std::cout << current->data << " ";
127         current = current->prev;
128     }
129     std::cout << std::endl;
130 }
131 
132 void DoublyLinkedList::clear() {
133     while (head) {
134         Node* temp = head;
135         head = head->next;
136         delete temp;
137     }
138     tail = nullptr;
139     size = 0;
140 }
141 
142 size_t DoublyLinkedList::getSize() const {
143     return size;
144 }
145 
```

```
146 ListIterator DoublyLinkedList::getIterator() const {
147     return ListIterator(head);
148 }
149
150 ListIterator DoublyLinkedList::getReverseIterator() const {
151     return ListIterator(tail);
152 }
```

Listing 7. Plik źródłowy DoublyLinkedList.cpp

4.5. Szczegółowa analiza kluczowych fragmentów kodu

4.5.1. Wstawianie pod dowolny indeks (`insertAt`)

Warto zwrócić uwagę na sposób przepinania wskaźników podczas dodawania elementu wewnątrz listy w metodzie `insertAt`:

```
Node* newNode = ListFactory::createNode(value);
Node* prevNode = current->prev;

prevNode->next = newNode;
newNode->prev = prevNode;

newNode->next = current;
current->prev = newNode;
```

Kolejność aktualizacji wskaźników ma kluczowe znaczenie. Przypisanie `newNode->prev = prevNode` oraz `newNode->next = current` gwarantuje, że nowy węzeł ma prawidłowo ustawione wskaźniki przed zmodyfikowaniem istniejących powiązań w liście.

4.5.2. Bezpieczne zwalnianie pamięci w `popFront` i `clear`

W metodzie `popFront()` usunięcie węzła odbywa się poprzez wcześniejsze zapamiętanie wskaźnika w zmiennej tymczasowej `temp`:

```
Node* temp = head;
head = head->next;
if (head) {
    head->prev = nullptr;
} else {
    tail = nullptr;
}
delete temp;
```

Zapobiega to przypadkowemu odwołaniu się do usuniętego obszaru pamięci (tzw. *use-after-free*).

4.6. Plik wykonawczy main.cpp

W pliku `main.cpp` zaimplementowano scenariusze weryfikujące poprawność działania wszystkich napisanych funkcji.

```
#include <iostream>
#include "DoublyLinkedList.h"

int main() {
    DoublyLinkedList list;

    std::cout << "--- TESTOWANIE LISTY DWUKIERUNKOWEJ ---" << std::
   endl;

    list.pushBack(10);
    list.pushBack(20);
    list.pushFront(5);
    list.displayForward();

    list.insertAt(1, 7);
    list.displayForward();

    list.displayBackward();

    list.popFront();
    list.displayForward();

    list.popBack();
    list.displayForward();

    list.pushBack(30);
    list.pushBack(40);
    list.displayForward();

    list.removeAt(2);
    list.displayForward();

    std::cout << "\n--- TEST ITERATORA ---" << std::endl;
    ListIterator it = list.getIterator();
    std::cout << "Aktualny (poczatek): " << it.get() << std::endl;
    if (it.hasNext()) {
        std::cout << "Nastepny: " << it.next() << std::endl;
```

```
37      }
38      if (it.hasPrev()) {
39          std::cout << "Poprzedni: " << it.prev() << std::endl;
40      }
41
42      list.clear();
43      std::cout << "Po wyczyszczeniu:" << std::endl;
44      list.displayForward();
45
46      return 0;
47  }
```

Listing 8. Plik główny main.cpp

5. Wnioski i podsumowanie

5.1. Podsumowanie wykonanych prac

W ramach zrealizowanego projektu zaprojektowano i pomyślnie zaimplementowano pełną bibliotekę dynamicznej listy dwukierunkowej w języku C++. Zgodnie z przyjętymi założeniami programistycznymi, system został podzielony na czytelne moduły:

- Węzeł danych (`Node`),
- Wzorzec fabryki (`ListFactory`),
- Zewnętrzny iterator (`ListIterator`),
- Główną strukturę danych (`DoublyLinkedList`).

Przeprowadzone testy w pliku `main.cpp` potwierdziły poprawność działania wszystkich zaimplementowanych operacji, takich jak dodawanie na krańcach, wstawianie pod dowolny indeks, usuwanie wybranych węzłów, dwukierunkowe wyświetlanie oraz przeglądanie listy za pomocą iteratora.

5.2. Ocena realizacji celów i wymagań

Wszystkie wymagania funkcjonalne (WF-01 do WF-09) opisane w Rozdziale 1 zostały w pełni zrealizowane. Kod programu jest wolny od wycieków pamięci, co zapewniono poprzez skrupulatne czyszczenie zasobów w destruktorze oraz metodzie `clear()`.

Zastosowanie wzorców projektowych pozwoliło stworzyć kod modułowy i podatny na rozbudowę. Użycie systemu kontroli wersji Git oraz platformy GitHub zapewniło przejrzystość historii rozwoju projektu oraz ułatwiło zarządzanie wersjami kodu źródłowego.

5.3. Możliwości dalszego rozwoju projektu

Pomimo że zrealizowana biblioteka stanowi w pełni funkcjonalne rozwiązanie, projekt posiada potencjał do dalszej rozbudowy o następujące funkcjonalności:

1. **Uogólnienie typów (Szablony / Templates):** Przekształcenie klasy z operującej wyłącznie na typie `int` na klasę szablonową `DoublyLinkedList<T>`, co umożliwi przechowywanie dowolnych typów danych.

2. **Smart Pointers:** Zastosowanie inteligentnych wskaźników z biblioteki standardowej (`std::shared_ptr`, `std::unique_ptr`, `std::weak_ptr`) w celu automatyzacji zarządzania czasem życia obiektów i uniknięcia jawnych wywołań `delete`.

3. **Zgodność z C++ Standard Template Library (STL):** Dostosowanie iteratora do wymagań `std::iterator_traits`, co pozwoli na stosowanie algorytmów z nagłówka `<algorithm>` (np. `std::sort`, `std::find`).

4. **Wielowątkowość:** Dodanie mechanizmów synchronizacji (np. `std::mutex`) pozwalających na bezpieczny dostęp do struktury danych z poziomu wielu równoległych wątków.

5.4. Uwagi końcowe

Stworzona dokumentacja oraz kod źródłowy stanowią kompletny projekt zaliczeniowy z przedmiotu Zaawansowane Programowanie, demonstrujący wiedzę z zakresu struktur danych, zarządzania pamięcią operacyjną oraz dobrych praktyk inżynierii oprogramowania.

Spis rysunków

Spis tabel

1.1. Zestawienie wymagań funkcjonalnych systemu 6

2.1. Porównanie złożoności obliczeniowej operacji na liście dwukierunkowej 7

3.1. Specyfikacja interfejsu klasy `DoublyLinkedList` 10

Spis listingów

1. Plik nagłówkowy Node.h 11
2. Plik źródłowy Node.cpp 11
3. Plik nagłówkowy ListFactor.h 11
4. Plik źródłowy ListFactor.cpp 12
5. Plik nagłówkowy ListIterator.h 12
6. Plik źródłowy ListIterator.cpp 12
7. Plik źródłowy DoublyLinkedList.cpp 14
8. Plik główny main.cpp 19